Algebraic World Model: From Observations to Axioms and Proofs

Anonymous authors
Paper under double-blind review

Abstract

AI has made remarkable progress in mathematics, solving challenging problems with machine-checkable proofs in Lean, yet typically relying on predefined mathematical definitions and formal specifications. This raises a natural question: can it reason rigorously about an unknown world? We introduce the Algebraic World Model (AWM), which turns interaction with an unfamiliar environment into a mathematical model of it. The agent explores the environment and prototypes executable hypotheses, revising them through each transition. It then structures these hypotheses into typed objects and shared rules, simplifies them through category-theory-inspired transformations, and formalizes the resulting model in Lean 4 to prove reachability and impossibility. On two impossible game levels, AWM agents construct closed, kernel-checked Lean proofs that exclude every legal winning sequence under explicit learned models in all six runs. Without AWM, Claude Opus 5.5 and GPT-6 Astra produce no such certificates in twelve runs; even the sole numerically correct bound leaves route minimality as an explicit, unchecked assumption. AWM also substantially improves bound accuracy to 4/6, compared with 1/6 for Opus 5.5 and 0/6 for GPT-6 Astra, and reaches 100% across the six runs after feedback-guided repair. AWM also models all 25 public ARC-AGI-3 games: Lean replays the winning runs of all 183 levels, and with Opus 5 it scores 99.9 with the fewest certified actions among the systems that report them.

1 Introduction

Artificial intelligence has made remarkable progress in mathematical reasoning: automated systems now produce Lean proofs of olympiad problems and of open Erdős problems (Achim et al., 2025; Hubert et al., 2026; Sothanaphan, 2026; Tsoukalas et al., 2026). These advances invite a broader question:

Can an agent reason rigorously about an unknown world?

In formal mathematics the definitions needed to state a problem are given in advance, and a proof checker verifies whether a conclusion follows from them. In an unfamiliar environment the agent must first discover the relevant objects, infer how actions transform them and determine what counts as success; the rules required for formal reasoning are themselves hypotheses about the world, learned from experience and open to error.

Euler's treatment of the Königsberg bridges shows what such a structure can provide (Euler, 1741). Land masses become vertices and bridges become edges; a traversal pairs arrivals with departures at every intermediate vertex, so at most two vertices can have odd degree, and the four odd-degree vertices of the Königsberg graph exclude it. The graph rules out every candidate walk, including the walks nobody tried. Finding one winning route shows that a goal can be reached; showing that none exists needs a model of this kind, and in an unfamiliar environment that model has to be learned.

The Algebraic World Model (AWM) builds such a model through three stages (Figure 1). **Explore & Prototype:** the agent actively explores the environment, building and iteratively revising a provisional executable model from observed transitions. **Structure & Simplify:** it organizes this prototype into typed objects and rules, then applies category-theory-inspired transformations to discover and simplify their underlying structure while preserving empirically tested distinctions. **Formalize & Prove:** it encodes the resulting model in Lean 4 (de Moura & Ullrich, 2021) and constructs

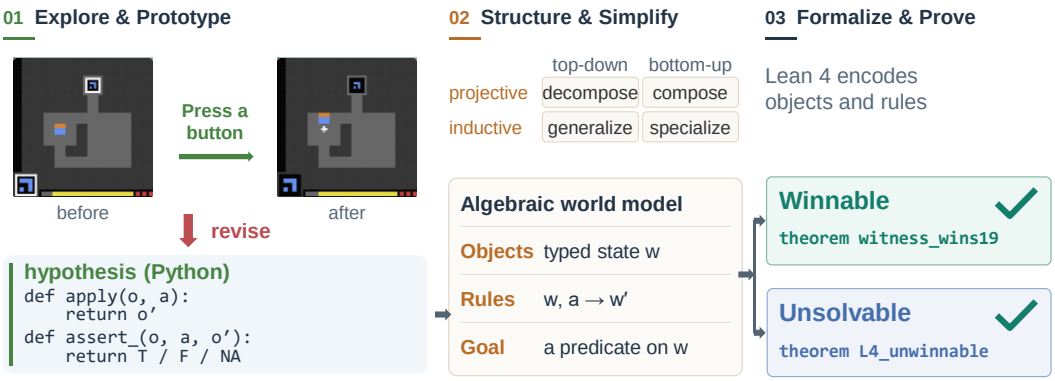


Figure 1: AWM's three-stage pipeline: interactive exploration builds a provisional executable model, algebraic transformations reveal and simplify its structure, and Lean 4 formalization enables certified proofs of reachability or impossibility.

closed, kernel-checked proofs of reachability or impossibility. A winning witness certifies reachability, while an invariant or cost bound can rule out all legal winning sequences under the model, including those never explored by the agent. Unlike conclusions drawn from finite exploration alone, such impossibility proofs cover all behaviors permitted by the inferred model, whose premises remain open to empirical testing.

ARC-AGI-3 (ARC Prize Foundation, 2026b) makes this challenge concrete: its games reveal neither their rules nor their goals. We seal two `sc25` levels one unit below their minimum winning cost and ask whether any legal winning sequence remains (§4). In all six runs, AWM proves in Lean that none remains under its learned model, while Claude Opus 5.5 and GPT-6 Astra produce no such proof in twelve runs; on level 4, GPT-6 Astra puts the minimum cost at 22, where AWM proves the engine's 19. AWM's bounds match the engine's optimum in four of six runs, against one for Opus 5.5 and none for GPT-6 Astra, and in all six after a generic feedback signal, each with a new formal proof. The models behind these proofs are learned from play alone: AWM builds models of all 25 public games, and Lean replays the winning runs of all 183 levels against them (§3).

We make three contributions.

- **Algebraizing an unknown world.** A three-stage method that transforms interaction into executable hypotheses, structures them into compact algebraic world models through category-theory-inspired transformations, and formalizes them in Lean 4 for machine-checked reasoning (§2).
- **Formal impossibility reasoning.** In all six runs on two sealed levels, AWM constructs closed, kernel-checked Lean proofs that rule out every legal winning sequence under its learned models; Claude Opus 5.5 and GPT-6 Astra produce no such proof, and GPT-6 Astra's bound of 22 would wrongly rule out wins that cost 19 to 21 (§4).
- **Algebraic models of 25 games.** AWM models all 25 public ARC-AGI-3 games, and Lean formally replays winning trajectories across all 183 levels. With Opus 5, it achieves a score of 99.9 and the fewest certified actions among reporting systems (§3).

2 Method

AWM aims to turn interaction with an unknown environment into a mathematical model that supports rigorous reasoning about both reachability and impossibility. This requires more than collecting rules that explain observed transitions: the rules must form a coherent representation of the environment, and that representation must support reasoning about action sequences the agent has never explored.

We first describe the target model (§2.1), then present the three stages that construct and use it (Figure 1): Explore & Prototype develops executable hypotheses through interaction; Structure & Sim-

plify organizes them into a compact algebraic model; and Formalize & Prove translates the model into Lean 4 and establishes machine-checked results.

2.1 The model: objects, rules, and formal representation

An algebraic world model represents an environment through typed objects, rules, and a goal predicate, with executable Python and formal Lean 4 representations supporting empirical testing and rigorous reasoning.

Objects. An object type specifies structured data and its validity conditions; an object is a value of that type. A parser maps observations to typed states w, including history or auxiliary information when the current observation alone is insufficient.

Rules. A rule specifies when a mechanism applies and how it behaves. Its Python representation defines an applicability scope and a checker that reports agreement, contradiction, or non-applicability on a transition; effect rules also predict all or part of the successor state. Together, the rules specify legal actions, their effects and costs $\text{cost}(w, a)$, while the goal predicate identifies successful states.

Related mechanisms can share parameterized operations $r(\theta)$, with parameters determined by the state. This structure allows common properties to be expressed and proved across multiple instances rather than separately for each rule.

Formal representation. The same model is also represented in Lean 4, where objects and rules become formal definitions and properties of their dynamics become theorems. Python supports empirical testing against observations, while Lean supports kernel-checked reasoning about legal action sequences under the model. The formal representation is constructed in §2.4.

2.2 Explore & Prototype

Setting. We consider deterministic, goal-directed environments with partial observations, modeled in the POMDP setting (Kaelbling et al., 1998). At step t, the agent observes o_t, takes an action a_t, and receives o_{t+1} without access to the underlying state or transition function. Its experience forms a corpus $\mathcal{D}_t$ of distinct transitions $x = (o, a, o')$.

Building and testing hypotheses. AWM actively explores the environment and maintains a provisional executable model as a *concept book*. Each concept is a Python hypothesis about an environmental mechanism, containing a natural-language claim, an applicability scope, supporting transitions, a checker, and an optional predictor.

For a concept c, its checker returns $v_c(x)$: `TRUE` if the transition agrees with the hypothesis, `FALSE` if it contradicts it, and `NA` if it falls outside its scope. The optional predictor specifies expected action effects for comparison with subsequent observations.

After each action batch, AWM rechecks the concept book $\mathcal{C}_t$ against the expanded corpus $\mathcal{D}_t$. Further interaction is allowed only when

$$\forall c \in \mathcal{C}_t,\ \forall x \in \mathcal{D}_t : \quad v_c(x) \notin \{\mathsf{FALSE}, \mathsf{ERR}\}, \tag{1}$$

where ERR denotes checker failure, including exceptions and timeouts.

This recheck ensures that contradicted or failing concepts must be revised or withdrawn before exploration continues. The agent then keeps revises and uses its hypotheses to plan toward inferred goals or probe competing explanations.

The resulting prototype captures tested mechanisms but may describe related behaviors independently, motivating the next stage.

2.3 Structure & Simplify

The concept book may contain fragmented, overlapping descriptions of environmental mechanisms. Structure & Simplify turns these hypotheses into a compact model by identifying shared structure and reorganizing components through their relationships.

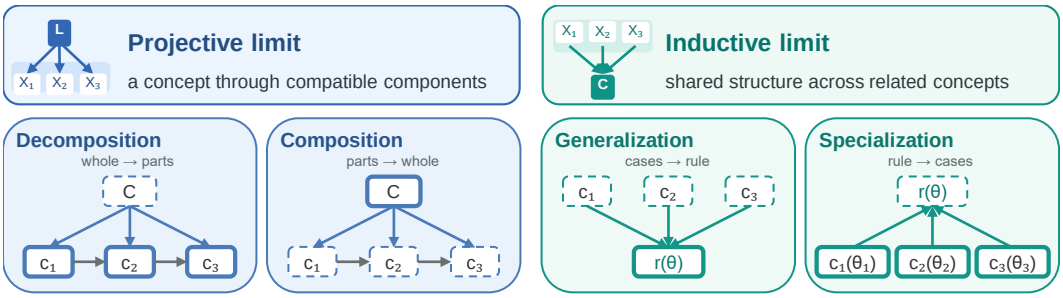


Figure 2: The four abstraction transformations, grouped by the limit they follow; each applies to object types and to rules. Arrows follow the limit; a dashed box is removed by the transformation and a bold box is created.

AWM first extracts the required object types, constructs a parser from observations to typed states, and rewrites concepts as rules over these objects. It then simplifies the resulting *components*—object types and rules—through four transformations inspired by category theory. In category theory, a *projective limit* assembles compatible descriptions into a coherent whole, while an *inductive limit* glues objects along shared structure (Mac Lane, 1998; Leinster, 2014; Yuan, 2023). These dual perspectives guide four transformations (Figure 2).

Following projective limits, *decomposition* expresses a complex component through simpler components and their compatibility relations. Conversely, *composition* identifies components determined by compatible existing ones, allowing redundant definitions to be removed.

Following inductive limits, *generalization* extracts shared structure from multiple components, expressing source rules c_i through a parameterized operation $r(\theta_i)$ with state-dependent θ_i. Conversely, *specialization* refines overly general components into more specific cases or introduces parameters to reduce exceptions and redundancy.

Preserving tested distinctions. Every transformation must preserve verdicts on recorded transitions. For rules, AWM also evaluates a diagnostic set $\mathcal{N}$ of perturbed recorded transitions, most of them invalid under the environment's dynamics, to prevent simplification from losing previously detected rejections.

Define $\text{Rej}(c) = \{x \in \mathcal{N} : v_c(x) = \textsf{FALSE}\}$ and $\text{Rej}(S) = \bigcup_{c \in S} \text{Rej}(c)$. Replacing rules S with T must preserve coverage of recorded transitions and satisfy

$$\text{Rej}(T) \supseteq \text{Rej}(S). \tag{2}$$

For object types, which have no checkers, transformations are evaluated by distinctions between recorded states, using state pairs as witnesses, and by rule verdicts once rules are available.

AWM applies these transformations and tests to object types, then rules, and finally rechecks object types against the settled rules. Together, these transformations restructure the model to reduce its representational complexity. The resulting compact structure provides a common representation for formal reasoning.

2.4 Formalize & Prove

AWM completes the model in Lean 4, encoding object types as structures or inductive types, rules as `def` definitions, and derived properties as `theorem` statements. The transition function dispatches to named rules, while unresolved assumptions remain explicit theorem hypotheses rather than unchecked `axiom` declarations. Replayed along the recorded winning runs, this compact algebraic model reproduces the observed behavior on every component it models, so its theorems concern the same dynamics the agent played, now in a form where claims about every action sequence can be proved.

Proving reachability and impossibility. A level is *winnable* in the model if a legal action sequence reaches the goal. Lean proves reachability by checking such a witness. To prove impossibility, AWM

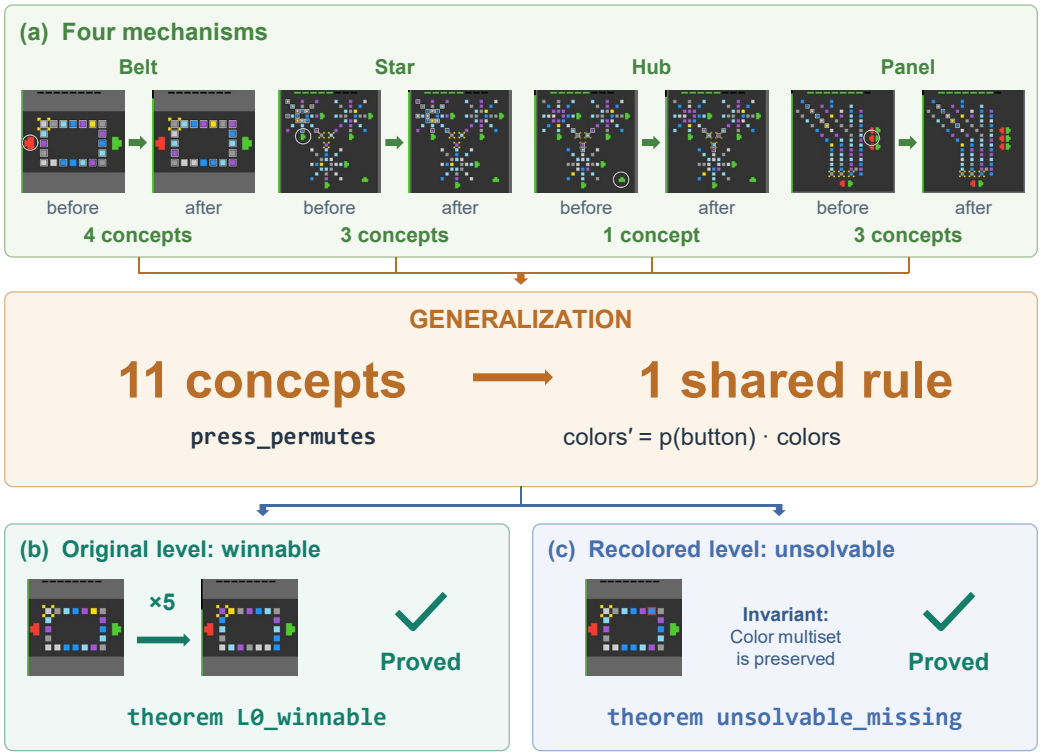


Figure 3: From shared structure to formal proofs in `lp85`. (a) Eleven concepts describing four button mechanisms are generalized into a shared permutation rule. Each before/after pair shows a recorded press, with the pressed button circled and changed tiles outlined. (b) A five-click sequence witnesses reachability. (c) Recoloring the only yellow tile makes the goal unreachable, since every press preserves color counts. Both conclusions are formally proved in Lean 4.

instead rules out all legal winning sequences, including unexplored ones, using an invariant that excludes the goal or a cost lower bound. For the latter, a potential V satisfies

$$V(w) \leq \text{cost}(w, a) + V(w') \quad \text{for every legal transition,} \qquad V \leq 0 \quad \text{on the goal,} \tag{3}$$

so every winning sequence costs at least $V(w_0)$, proving impossibility when this exceeds the budget. All proofs are closed and checked by the Lean kernel without `sorry` or `native_decide`, establishing rigorous guarantees under the learned model's explicit premises.

2.5 Example: one rule for four mechanisms

In game `lp85`, buttons move colored tiles, and a level is won when every slot holds its matching color. During Explore & Prototype, the agent discovers eleven concepts describing four button mechanisms: belts, stars, hubs, and panels (Figure 3a). In Structure & Simplify, AWM applies the generalization transformation to recognize that all four mechanisms permute tile positions, replacing the eleven concepts with a shared operation, `press_permutes`, parameterized by the pressed button. The unified rule preserves all 55 diagnostic rejections of the original concepts and detects 21 additional invalid transitions.

In Formalize & Prove, Lean certifies a five-click winning sequence (Figure 3b) and proves a recolored variant unwinnable using the color-count invariant shared by all button permutations (Figure 3c). The final model contains six object types and six rules, and Lean replays all 199 recorded transitions without mismatch.

Table 1: Official scores on the 25 public ARC-AGI-3 games. Claude Opus 5 and GPT-6 Astra run under the official standard harness, which provides no code execution; the other systems execute code. AWM's certified clears follow uncertified sessions of the same run in 22 games. baseline1's actions are from its ARC Prize scorecard and Memento 3's as reported in (Zhao et al., 2026).

System	Code	Mean score	Games at 100	Certified actions	Actions / human
Claude Opus 5	no	40.7	5	—	—
GPT-6 Astra	no	68.3	10	—	—
GPT-6 Astra (adapter) (ARC Prize Foundation, 2026a)	yes	100.0	25	—	—
baseline1 (Rodionov, 2026b)	yes	99.0	23	8,347	0.49
Memento 3 (Zhao et al., 2026)	yes	100.0	25	7,518	0.44
AWM (ours)	yes	99.9	23	**7,448**	**0.43**

Table 2: Formalization and verification of the 25 learned world models in Lean 4.

Levels replayed	Rules as Lean definitions	Theorems	Axioms
183 / 183	220 / 283	1,372	only `propext`, `Classical.choice`, `Quot.sound`

3 Twenty-five public games

ARC-AGI-3 (ARC Prize Foundation, 2026b) scores each level by the squared ratio of a human baseline's action count to the agent's, weights later levels more, and gives an uncleared level zero. We evaluate AWM with Claude Opus 5 on all 25 public games (183 levels), starting each game from scratch. Beyond game performance, we examine whether the resulting models are compact, reproduce observed behavior in Lean, and generalize to unseen transitions.

Playing efficiently. AWM with Opus 5 achieves a score of 99.9, compared with 40.7 for Opus 5 under the official standard harness without code execution. It scores 100 on 23 games and at least 97.6 on every game, matching the performance of leading code-executing systems (Table 1). Its certified clears require 7,448 actions, only 43% of the human baseline's 17,135 and the fewest among reporting systems, compared with 7,518 for Memento 3 (Zhao et al., 2026) and 8,347 for baseline1 (Rodionov, 2026b). These results show that AWM can construct and use explicit world models while retaining competitive action efficiency.

Compact models that reproduce observed behavior. Across the 25 games, AWM develops 742 executable concepts, withdrawing 143 during exploration. Structure & Simplify organizes them into 170 object types and 283 rules, with a median of 6 types and 10 rules per game. Shared patterns recur across games, including translations, permutations, and unified goal conditions, illustrating how different mechanisms can be expressed through common structures.

The resulting models agree with 23,652 of 23,961 recorded transitions (98.7%), including exploration.[1] Only 52 transitions involve incorrect predictions; most remaining cases receive no verdict because of history-dependent or unobserved state information, or reset and level-change behavior. In Formalize & Prove, AWM constructs Lean models that replay the winning trajectories of all 183 levels from their initial states and verify their final states against the modeled goals (Table 2).

Generalization beyond observed transitions. Agreement with recorded play alone does not establish whether learned rules capture unseen behavior. We therefore freeze the models of `lp85` and `ft09` after exploration and evaluate them on transitions from random play with previously unseen screens (seeds 0–9; detailed results in the supplementary material). On these trajectories, the frozen rules accept all valid transitions, reject at least 99.5% of corrupted ones, and predict successor tile layouts with at least 99.95% accuracy, including previously unseen layouts in `lp85`.

We further modify `ft09`'s hidden click mechanism, causing each click to change 110 or 146 cells instead of 38. The frozen rules reject every resulting transition, demonstrating that they can detect

[1] For most games, agreement means that the Lean model's replay matches the record on the components it represents; for `s5i5` and `m0r0`, it means no misjudgment at the rule level.

Table 3: Impossibility reasoning across six runs per agent. Bound accuracy is evaluated separately against engine optima, before/after feedback-guided repair. First claim averages the earliest impossibility claim among runs making one, including later withdrawals; it is not proof-completion time.

Agent	Kernel-checked model proofs	Bounds matching optimum initial → feedback	Ever claimed impossible	First claim (attempt)
Claude Opus 5.5	0/6	1/6 → 1/6	3/6	5.3
GPT-6 Astra	0/6	0/6 → 2/6	2/6	8.0
AWM	**6/6**	**4/6 → 6/6**	**6/6**	**2.0**

behavior inconsistent with the learned dynamics. Together, these tests provide evidence that AWM's rules capture reusable mechanisms beyond the trajectories from which they were learned. §4 next examines whether such models can support formal reasoning about impossible goals.

4 Proving impossibility

A winning trajectory establishes reachability; impossibility requires excluding every legal winning sequence within the budget, including those never explored. We test whether AWM can turn learned rules into such proofs, measuring model-relative proof validity separately from numerical agreement with the game.

4.1 The game, the seal, and the baselines

In `sc25`, a block moves on a grid and casts spells through a 3×3 panel. Moves and panel clicks cost one unit, completing a spell one more, fuel refunds ten, and the winning move is free. We construct two modified levels based on levels 4 and 5 by rearranging selected elements while preserving the original game mechanics, then seal them with action budgets of 18 and 35. Exhaustive engine search establishes minimum winning costs of 19 and 36, respectively: neither sealed level can be won, while one additional unit permits a win. An impossibility verdict can be justified by a lower bound on the minimum winning cost that exceeds the sealed budget. We evaluate the numerical agreement of reported bounds with the engine optima separately from their formal validity under the learned models.

Claude Opus 5.5 and GPT-6 Astra receive the game interface and coding tools, including simulator construction and search. AWM uses Opus 5.5 with the procedure of §2 and a generic Lean skeleton. We evaluate three runs per agent per level, allowing up to eight attempts per run; an attempt ends at game over or abandonment. All agents are asked for a verdict and supporting justification; AWM thus constructs formal proofs through its learned Lean model.

4.2 Certified lower bounds

Proving beyond explored trajectories. All six AWM runs yield closed, kernel-checked impossibility proofs under their learned models and explicit premises; none of the twelve runs without AWM produces such a certificate (Table 3). An independently rebuilt L4 development, for example, proves

$$\forall B \le 18,\ \forall \mathbf{a}, \quad \mathrm{Won}(\mathrm{run}_B(w_0, \mathbf{a})) = \mathsf{false}, \tag{4}$$

and checks a winning witness at $B = 19$. The lower bound excludes every cheaper modeled sequence, while the witness establishes attainability, together certifying a tight threshold in that model. A winning route alone would not exclude cheaper alternatives.

AWM establishes such bounds through potential or closure certificates over arbitrary legal action lists. External search may propose certificate data, but the resulting optimality claim is verified in Lean rather than trusted from the search. The proofs explicitly expose their modeled rules and premises, allowing proof validity to be assessed separately from agreement with the game.

Beyond unproved numerical judgments. Only five of twelve baseline runs claim impossibility, four with incorrect bounds. Even the sole correct Claude Opus 5.5 bound lists "that my hand-counted

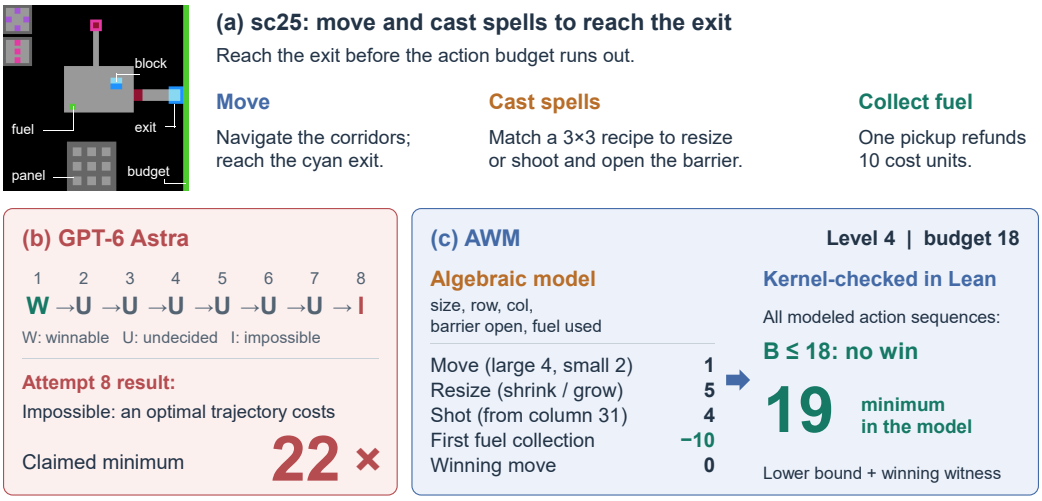


Figure 4: Sealed `sc25` level 4. (a) GPT-6 Astra estimates a minimum winning cost of 22 after eight attempts, versus the engine's 19. (b) AWM constructs a kernel-checked lower-bound proof and a matching winning witness at 19 under its learned model.

path lengths are truly minimal" as *assumed, not checked*. The missing step is a global lower bound: checking a route does not exclude cheaper alternatives. AWM discharges this obligation through kernel-checked proofs under its learned models.

Proof validity and bound accuracy are distinct. Four of six AWM bounds agree with the engine optima before feedback, compared with one for Opus 5.5 and none for GPT-6 Astra. We count a bound as correct if it matches the engine optimum or differs only by explicitly charging the otherwise free winning move; this is a cost-accounting convention, not a numerical tolerance. The two incorrect AWM bounds, 20 on L4 and 47 on L5, follow from overly restrictive learned rules: their proofs are valid for those models, but their thresholds do not describe the game correctly. Figure 4 contrasts Astra's incorrect estimate with a tight, formally proved model bound.

4.3 Auditing premises and proving again

After the initial evaluation, runs with incorrect bounds receive a generic signal that the engine permits a cheaper win, without the correct bound, route or faulty rule being named. AWM relaxes its explicit rules, tests the resulting disagreements through interaction and proves the refined conclusions again.

On L4, growing onto fuel is found to collect it, reducing the bound from 20 to 19. On L5, a shot's failure is traced to terrain rather than limited range, reducing 47 to 37 with the winning move charged (36 otherwise). Both repairs succeed within four additional attempts and yield independently rebuilt model proofs. With additional feedback, AWM reaches the correct bounds in all six runs, with each refined conclusion supported by a new formal proof.

We provide the same discrepancy feedback and additional attempt budget to the four baseline runs with incorrect bounds. Astra corrects both of its bounds, but neither produces a formal proof, while Claude Opus 5.5 fails to correct either. We additionally conduct exploratory autonomous repair runs to investigate whether AWM can identify and refine problematic rules and recover correct bounds without external discrepancy feedback; results are reported in the supplementary material.

5 Related work

Executable world models. Code world models are learned from interaction for planning (Tang et al., 2024), refined by generation, testing and repair (Dainese et al., 2024) or composed from programmatic experts (Piriyakulkij et al., 2025); theory-based agents explore and plan with learned theories (Tsividis et al., 2021), and TheoryCoder-2 learns reusable abstractions for hierarchical plan-

ning (Ahmed et al., 2026). Such models are central to recent ARC-AGI-3 agents (Rodionov, 2026a; Courtis et al., 2026; Rodionov, 2026b): Twin does not act until its simulator reproduces every observed transition (Skoutnev et al., 2026), and Tycho finds that a closer match to observed transitions does not by itself yield better play (Lehmann et al., 2026). The closest work, Memento 3, compiles a natural-language rulebook into code and accepts an update only if an LLM judges it faithful and replay is exact (Zhao et al., 2026). As it notes, replay leaves models that disagree on unobserved states; indeed, a code world model can pass a sampled-transition check at 100% accuracy and still lose because it misses a pivotal rule (Aguilar Martín, 2026). AWM instead abstracts its rules into a shared, named structure, tests them beyond the record, and proves model-relative reachability and impossibility.

Certified impossibility. Unsolvability certificates let an independent checker confirm that a planning task has no plan (Eriksson et al., 2017), in the spirit of certifying algorithms (McConnell et al., 2011), and certificate checking has been formally verified for temporal planning (Wang & Abdulaziz, 2025); all start from a supplied model. Impossibility transfers from an abstraction only if it over-approximates the real system, budget semantics included (Clarke et al., 1994), and action-model learning gives conditions under which learned models do so (Aineto & Scala, 2024). AWM learns its model from interaction; Lean checks deductions relative to that model, not the truth of its rules, which stay open to testing.

Formal reasoning. Neural provers produce Lean proofs at olympiad level (Achim et al., 2025; Chen et al., 2025; Hubert et al., 2026; Ren et al., 2025), AI systems have produced Lean proofs of open Erdős problems (Sothanaphan, 2026; Tsoukalas et al., 2026), and autoformalization translates informal statements into formal ones (Wu et al., 2022). There the objects of proof are given mathematics; AWM must first infer the dynamics and costs its proofs quantify over, which makes the fidelity of its formal model an empirical question.

6 Limitations and conclusion

Limitations. Our current framework relies on ARC-like environments with discrete actions and immediate, observable feedback, where learned rules can be checked against recorded transitions. Extending this process to real-world settings with partial observations, delayed feedback, or stochastic dynamics remains an open challenge. While Lean guarantees the correctness of proofs under the learned model's explicit premises, establishing that these premises faithfully capture the environment remains an empirical challenge. Importantly, the explicit rules and kernel-checked proofs separate errors in the learned model from errors in formal reasoning, making discrepancies easier to trace to their assumptions and address through further exploration. Our quantitative evaluation of impossibility is limited to two sealed levels of one game, with additional environmental feedback used for model refinement. Demonstrating the broader benefits of such formal reasoning across diverse environments and tasks remains future work.

Conclusion. Formal reasoning traditionally begins with predefined objects and rules. We introduce a framework that instead discovers them through interaction and organizes them into an algebraic world model. Inspired by categorical limits and colimits, its abstraction stage unifies related mechanisms through their shared structure, producing compact, reusable rules rather than isolated descriptions of observations. Formalized in Lean, the resulting model supports machine-checked proofs of reachability and impossibility, including claims about action sequences never explored. By making the assumptions behind these proofs explicit, the framework also enables incorrect models to be examined and revised through further interaction.

Reproducibility statement

The readings of §2 and §3 come from the benchmark repository at commit `f607dfc0` (built 2026-10-17). Each game's model, tests and Lean project are regenerated by one command per game; the per-game Lean projects are checked with Lean 4.34.0 and do not use Mathlib. The readings of §4 come from the archive of the budget-sealed instance: both engine versions with their md5, the abstract model and its cross-check scripts with their raw outputs, the Lean files with their compile

logs and axiom audits, every file each agent wrote, and the verbatim end-of-attempt interviews; the per-run readings are given in the supplementary material. The game environments are third party; official replays are linked rather than redistributed.

Statement on AI use

The agents studied in this paper are LLM agents, so AI use is part of the method. Generative AI was also used to draft and edit the text and to write the analysis scripts. All reported numbers come from script outputs over the recorded runs, not from model-generated text. The Lean proofs are checked by the Lean kernel, independently of any model.

References

Tudor Achim et al. Aristotle: IMO-level automated theorem proving. *arXiv preprint arXiv:2510.01346*, 2025.

Javier Aguilar Martín. When a verified world model still loses: Play-adequacy vs prediction-accuracy in LLM-synthesized code world models. *arXiv preprint arXiv:2607.14169*, 2026.

Zergham Ahmed, Kazuki Irie, Joshua B. Tenenbaum, Christopher J. Bates, and Samuel J. Gershman. Learning abstractions for hierarchical planning in program-synthesis agents. *arXiv preprint arXiv:2602.00929*, 2026.

Diego Aineto and Enrico Scala. Action model learning with guarantees. In *Proceedings of the 21st International Conference on Principles of Knowledge Representation and Reasoning (KR)*, pp. 801–811, 2024. doi: 10.24963/kr.2024/75.

ARC Prize Foundation. GPT-6 Astra: ARC-AGI results. `https://arcprize.org/results/openai-gpt-6-astra`, 2026a. Public Demo, 25 environments; accessed 2 October 2026.

ARC Prize Foundation. ARC-AGI-3: A new challenge for frontier agentic intelligence. *arXiv preprint arXiv:2603.24621*, 2026b.

Luoxin Chen, Jinming Gu, Liankai Huang, Wenhao Huang, Zhicheng Jiang, Allan Jie, Xiaoran Jin, Xing Jin, Chenggang Li, Kaijing Ma, et al. Seed-Prover: Deep and broad reasoning for automated theorem proving. *arXiv preprint arXiv:2507.23726*, 2025.

Edmund M. Clarke, Orna Grumberg, and David E. Long. Model checking and abstraction. *ACM Transactions on Programming Languages and Systems*, 16(5):1512–1542, 1994. doi: 10.1145/186025.186051.

David Courtis, Wenhao Li, and Scott Sanner. OPINE-World: Programmatic world modeling with ontology-error-prioritized interactive exploration for ARC-AGI-3. *arXiv preprint arXiv:2607.01531*, 2026.

Nicola Dainese, Matteo Merler, Minttu Alakuijala, and Pekka Marttinen. Generating code world models with large language models guided by Monte Carlo tree search. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, volume 37, pp. 60429–60474, 2024. doi: 10.52202/079017-1933.

Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pp. 625–635. Springer, 2021.

Salomé Eriksson, Gabriele Röger, and Malte Helmert. Unsolvability certificates for classical planning. In *Proceedings of the 27th International Conference on Automated Planning and Scheduling (ICAPS)*, pp. 88–97, 2017. doi: 10.1609/icaps.v27i1.13818.

Leonhard Euler. Solutio problematis ad geometriam situs pertinentis. *Commentarii academiae scientiarum Petropolitanae*, 8:128–140, 1741. Volume dated 1736; Eneström index E53.

Thomas Hubert, Rishi Mehta, Laurent Sartran, Miklós Z. Horváth, Goran Žužić, Eric Wieser, Aja Huang, Julian Schrittwieser, Yannick Schroecker, Hussain Masoom, et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651(8106):607–613, 2026. doi: 10.1038/s41586-025-09833-y.

Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998. doi: 10.1016/S0004-3702(98)00023-X.

Jens Lehmann, Andrei Aioanei, and Sahar Vahdati. Tycho: Active abstraction with programmatic world models for ARC-AGI-3. *arXiv preprint arXiv:2607.28287*, 2026.

Tom Leinster. *Basic Category Theory*, volume 143 of *Cambridge Studies in Advanced Mathematics*. Cambridge University Press, 2014.

Saunders Mac Lane. *Categories for the Working Mathematician*, volume 5 of *Graduate Texts in Mathematics*. Springer, 2nd edition, 1998.

Ross M. McConnell, Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. Certifying algorithms. *Computer Science Review*, 5(2):119–161, 2011. doi: 10.1016/j.cosrev.2010.09.009.

Wasu Top Piriyakulkij, Yichao Liang, Hao Tang, Adrian Weller, Marta Kryven, and Kevin Ellis. PoE-World: Compositional world modeling with products of programmatic experts. In *Advances in Neural Information Processing Systems 38 (NeurIPS)*, volume 38, 2025. doi: 10.52202/085713-0896.

Z. Z. Ren, Zhihong Shao, Junxiao Song, Huajian Xin, Haocheng Wang, Wanjia Zhao, Liyue Zhang, Zhe Fu, Qihao Zhu, Dejian Yang, Z. F. Wu, Zhibin Gou, Shirong Ma, Hongxuan Tang, Yuxuan Liu, Wenjun Gao, Daya Guo, and Chong Ruan. DeepSeek-Prover-V2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. *arXiv preprint arXiv:2504.21801*, 2025.

Sergey Rodionov. Executable world models for ARC-AGI-3 in the era of coding agents. In *Artificial General Intelligence: 19th International Conference, AGI 2026*, Lecture Notes in Computer Science. Springer, 2026a. arXiv:2605.05138.

Sergey Rodionov. Do coding agents need executable world models, simplification, and verification to solve ARC-AGI-3? *arXiv preprint arXiv:2607.15439*, 2026b.

Alexy Skoutnev, Kirill Acharya, Gaston Longhitano, Madeleine Udell, Kevin Ellis, and Iddo Drori. Twin: Playing an unknown game with a test-time digital twin. *arXiv preprint arXiv:2608.14490*, 2026.

Nat Sothanaphan. Resolution of Erdős problem #728: a writeup of Aristotle's Lean proof. *arXiv preprint arXiv:2601.07421*, 2026.

Hao Tang, Darren Key, and Kevin Ellis. WorldCoder, a model-based LLM agent: Building world models by writing code and interacting with the environment. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, volume 37, pp. 70148–70212, 2024. doi: 10.52202/079017-2243.

Pedro A. Tsividis, Joao Loula, Jake Burga, Nathan Foss, Andres Campero, Thomas Pouncy, Samuel J. Gershman, and Joshua B. Tenenbaum. Human-level reinforcement learning through theory-based modeling, exploration, and planning. *arXiv preprint arXiv:2107.12544*, 2021.

George Tsoukalas et al. Advancing mathematics research with AI-driven formal proof search. *arXiv preprint arXiv:2605.22763*, 2026.

David Wang and Mohammad Abdulaziz. Formally verified certification of unsolvability of temporal planning problems. *arXiv preprint arXiv:2510.10189*, 2025.

Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*, volume 35, 2022.

Yang Yuan. Succinct representations for concepts. *arXiv preprint arXiv:2303.00446*, 2023.

Haoyu Zhao, Zhengxu Yu, Zhiyuan He, Meng Fang, Rasul Tutunov, Haitham Bou-Ammar, Weilin Luo, and Jun Wang. Memento 3: Model-based recursive self-improvement through reflective rulebooks. *arXiv preprint arXiv:2610.11794*, 2026.